

Verificación de la resistencia ante intentos de inyección y alteración de datos

Se ejecutaron ataques reales contra el sistema web de gestión de calificaciones de la ECEME: inyección SQL por sus puntos de entrada y manipulación directa de la base de datos para intentar falsear notas ya selladas en la cadena de bloques.

Fecha: 23 de julio de 2026 a las 10:59 p. m. **Objetivo:** backend Express + MySQL (localhost:3001)

Pruebas ejecutadas: 17

Pruebas superadas

16 de 17

el sistema rechazó o detectó el ataque

Inyecciones SQL exitosas

0

todas las consultas van parametrizadas

Alteraciones no detectadas

0

la cadena delató cada modificación

Hallazgos por corregir

1

Control de acceso

Inyección SQL

6 / 6 superadas

Se enviaron cargas maliciosas a los puntos de entrada del sistema (login, buscador de la bitácora y parámetros de la URL) para intentar que la base de datos las ejecute como instrucciones.

1 Bypass de login con tautología

✓ RESISTE

Ataque `identificador y contraseña = ' OR '1'='1`
Se esperaba Acceso denegado (401)
Resultado HTTP 401 — Usuario no encontrado o clave incorrecta

2 Anular la verificación de contraseña con un comentario SQL

✓ RESISTE

Ataque `identificador = admin@eceme.mil.bo' --`
Se esperaba Acceso denegado (401)
Resultado HTTP 401 — Usuario no encontrado o clave incorrecta

3 Tautología en el buscador de auditoría

✓ RESISTE

Ataque `GET /api/admin/auditoria?q=' OR '1'='1`
Se esperaba 0 registros (el texto se busca literal, no se ejecuta)
Resultado HTTP 200 — 0 registro(s) devuelto(s)
Nota *Búsqueda de control sin resultados: 0 registro(s)*

4 UNION SELECT para extraer contraseñas

✓ RESISTE

Ataque `q=' UNION SELECT password FROM usuarios --`
Se esperaba 0 registros y ningún error de SQL expuesto
Resultado HTTP 200 — 0 registro(s), sin error de SQL

5 Payload destructivo DROP TABLE

✓ RESISTE

Ataque `'; DROP TABLE bloques; -- (en el buscador y en el login)`
Se esperaba La tabla bloques sigue existiendo y con los mismos registros
Resultado Tabla intacta: 3 bloques antes, 3 después

6 Inyección en un parámetro numérico de la URL

✓ RESISTE

Ataque `GET /api/reportes/acta/1 OR 1=1`
Se esperaba No se filtran datos de otras materias ni se expone un error de SQL
Resultado HTTP 404 — Materia no encontrada

Alteración de datos

7 / 7 superadas

Se atacó la integridad de las actas selladas, modificando la base de datos **por fuera del sistema**, con acceso directo a MySQL — el peor escenario posible. También se intentó alterar la bitácora de auditoría.

10 Estado inicial de la cadena de actas

✓ RESISTE

Ataque (ninguno – línea base)
Se esperaba Cadena válida
Resultado 3 bloques · válida: true

11 Cambiar una nota ya sellada, directo en la base de datos

✓ RESISTE

Ataque `UPDATE bloques SET acta_json = JSON_REPLACE(..., '$.registros[0].parcial_final', '95.00')`
Se esperaba La verificación detecta DATOS_ALTERADOS en el bloque 0
Resultado válida: false · primer error: bloque #0 (DATOS_ALTERADOS)
Nota *El atacante modificó la nota pero el hash guardado ya no coincide con el recalculado.*

12 Falsificar además el hash del bloque alterado

✓ RESISTE

Ataque `UPDATE bloques SET hash = SHA2(CONCAT(acta_json,'x'), 256) WHERE indice = 0`
Se esperaba El bloque sigue inválido y se rompe el enlace con el bloque siguiente
Resultado válida: false · bloque 0 válido: false · enlace del bloque 1 intacto: false
Nota *Para que cuadrara tendría que rehacer TODA la cadena posterior con la fórmula exacta del hash.*

13 Eliminar un bloque intermedio de la cadena

✓ RESISTE

Ataque `DELETE FROM bloques WHERE indice = 1`
Se esperaba La cadena queda rota (ENLACE_ROTO) en el bloque siguiente
Resultado válida: false · primer error: bloque #2 (ENLACE_ROTO)

14 Modificar o borrar un bloque desde la API

✓ RESISTE

Ataque `PUT /api/blockchain/0 · POST /api/bloques`

Se esperaba No existe ningún endpoint que edite o borre bloques (404)

Resultado `PUT /api/blockchain/0 → 404 · DELETE /api/blockchain/0 → 404 · POST /api/bloques → 404`

Nota *La cadena sólo admite AÑADIR bloques; no hay ninguna vía de escritura para editarlos.*

15 Borrar el rastro modificando la bitácora de auditoría

✓ RESISTE

Ataque `UPDATE auditoria SET detalle = '(rastro borrado)' WHERE id = 38`

Se esperaba La bitácora se reporta como alterada, señalando el registro

Resultado íntegra antes: true → después: false · registro #38 (DATOS_ALTERADOS)

Nota *La bitácora tiene su propia cadena de hashes, independiente de la de actas.*

16 Restauración del estado original

✓ RESISTE

Ataque `(se deshacen todas las alteraciones)`

Se esperaba Ambas cadenas vuelven a verificarse como íntegras

Resultado actas válidas: true (3 bloques) · bitácora íntegra: true (38 registros)

Credenciales

3 / 3 superadas

Se revisó cómo se guardan y se exponen las contraseñas y qué información filtran los mensajes de error.

7 La respuesta del login no expone la contraseña

✓ RESISTE

Ataque `Inspeccionar el JSON que devuelve POST /api/login`

Se esperaba El campo password no viaja al cliente

Resultado HTTP 200 — respuesta sin campo password

8 Contraseñas almacenadas con hash bcrypt

✓ RESISTE

Ataque	Leer directamente la columna usuarios.password en MySQL
Se esperaba	Ninguna contraseña legible en texto plano
Resultado	27 de 27 usuarios con hash bcrypt · 0 en texto plano
Nota	<i>Aun con acceso total a la base, las contraseñas no se pueden leer.</i>

9 El error de login no revela si el usuario existe

✓ RESISTE

Ataque	Comparar la respuesta de un usuario inexistente con la de una contraseña incorrecta
Se esperaba	Mismo código y mismo mensaje en ambos casos
Resultado	usuario inexistente: 401 "Usuario no encontrado o clave incorrecta" · clave incorrecta: 401 "Usuario no encontrado o clave incorrecta"
Nota	<i>Impide averiguar qué usuarios existen probando identificadores (enumeración).</i>

Control de acceso

0 / 1 superadas

Se comprobó si la API exige credenciales para atender peticiones a rutas de administración.

17 Acceder a un endpoint de administración sin autenticarse

✗ HALLAZGO

Ataque	GET /api/admin/auditoria sin ninguna credencial ni cabecera de sesión
Se esperaba	Acceso denegado (401 / 403)
Resultado	HTTP 200 — 38 registro(s) de la bitácora entregados sin autenticación
Nota	<i>La API no exige token de sesión: la protección de rutas vive sólo en el frontend.</i>

Conclusiones

La capa blockchain cumple su función

El ataque más fuerte que se puede montar contra este sistema es el que se hizo: entrar directamente a MySQL y cambiar una nota ya publicada, sin pasar por la aplicación. Aun así **ninguna alteración pasó inadvertida**. Modificar la nota rompe el hash del bloque; falsificar el hash rompe el enlace con el bloque siguiente; borrar un bloque rompe la cadena. Para que el fraude cuadrara, el atacante tendría que rehacer todos los bloques posteriores conociendo la fórmula exacta del hash — y lo mismo, por separado, con la bitácora de auditoría.

Las notas son, en la práctica, **inalterables sin dejar evidencia**: es exactamente lo que el proyecto se propuso demostrar.

Hallazgo — Acceder a un endpoint de administración sin autenticarse

Qué se comprobó: HTTP 200 — 38 registro(s) de la bitácora entregados sin autenticación

Por qué importa: la protección de rutas está implementada en el frontend (React decide qué ve cada rol), pero la API no la exige. Cualquiera que conozca la dirección del servidor puede consultar los datos sin pasar por la pantalla de acceso.

Cómo se corrige: emitir un token de sesión al iniciar sesión y exigirlo en un middleware de Express para las rutas `/api/admin/*` y demás endpoints sensibles, verificando además el rol. Es un cambio acotado al backend y **no toca la capa blockchain**.

Alcance real: el sistema opera en la red interna de la institución, y este hallazgo permitiría *leer o escribir* datos operativos — pero no falsear un acta ya sellada sin que la cadena lo delate, como demuestran las pruebas anteriores.

Metodología

- **Automatizada:** las 17 pruebas están en `seguridad.js` y se ejecutan con un solo comando; los resultados quedan en `resultados.json`.
- **Ataques reales:** no son simulaciones. Las cargas se enviaron al servidor en funcionamiento y las alteraciones se aplicaron con sentencias `UPDATE` y `DELETE` sobre la base de datos real.
- **Criterio de éxito:** una prueba se supera si el ataque es *rechazado* (inyección) o *detectado* (alteración). Detectar es suficiente: la cadena no impide que alguien con acceso a MySQL escriba, sino que garantiza que no pueda hacerlo sin dejar rastro.
- **Reversibilidad:** se respaldó la base con `mysqldump` antes de empezar y cada alteración se deshizo dentro de la misma prueba. Al terminar, la cadena de actas y la bitácora se verificaron íntegras.

Informe generado automáticamente a partir de `resultados.json` · Sistema web con tecnología blockchain para la gestión segura de calificaciones y evaluación docente · Escuela de Comando y Estado Mayor del Ejército.